# Python Programming
# Assignment 1

ADITYA AJIT GOWARI
Roll Number: **2506**

4th August 2026

**GitHub Repository:** Github Repository: patzer_adi/Python_Workspace

---

## Objective

Python Assignment of reading and exploring through experiments.

## 1 Variables

**What:** A variable is a symbolic name referred to a value stored in memory.

**Why:** Variables let the program store data and label the data to make the code.

**Where and How:** Everywhere in program variables are used. to store calculations and store in values and pass data between functions.

Example: `program = "Python"`, `age = 22`.

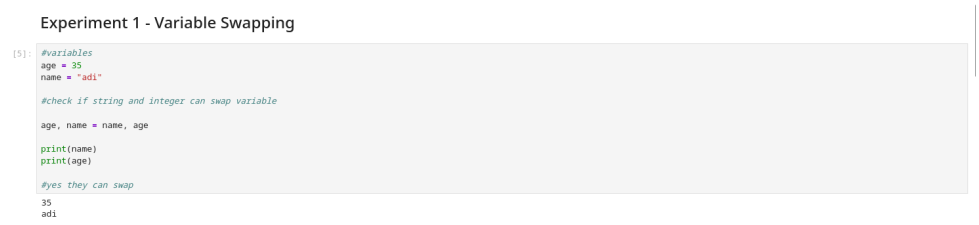



Figure 1: Output of Experiment 1

## 2 Basic Data Types

**What:** A datatype describes what kind of value do a variable hold.

**Why:** Data types are used in programming to tell the computer how to interpret, store, and manipulate data.

**Where/How:** Every python variable has a type and each type can be used in different ways. Explained below.

## 2.1 Integer

**What:** The `int` type represents integer values i.e +ve and -ve numbers, including zero but no fractional numbers.

**Why:** Integers are needed for counting, indexing, and any calculation where fractional precision is not required.

**Where and How:** Used for loop counters like counts of items, increment a loop variable, etc and perform and store arithmetic operations. Example: `-5`, `0`, `42`. Figure 1: age = 35 is an integer type.

## 2.2 Float

**What:** The `float` type represents real numbers that include a decimal point i.e a fractional number. (e.g. `3.14`, `-0.5`).

**Why:** Many real world quantities i.e prices of items, weight / height measurements or averages are not whole numbers they are fractions so we need a type understanding and representing fractions.

**Where and How:** Used in scientific calculations, any computation involving division. Example: `cgpa = 8.75`.

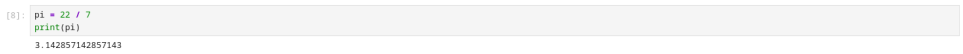



Figure 2: Output of pi

## 2.3 Boolean

**What:** The `bool` type has exactly two values `True` and `False`, and is technically a subtype of `int` (`True == 1`, `False == 0`). It acts as binary values.

**Why:** Programs constantly need to represent yes/no or on/off conditions, and booleans are the datatype that represent those binary values of comparisons and logical operations.

**Where and How:** Used in conditional statements (`if`), loop conditions (`while`), and as flags in code for safety checks.

Example: `is_passed = True`.

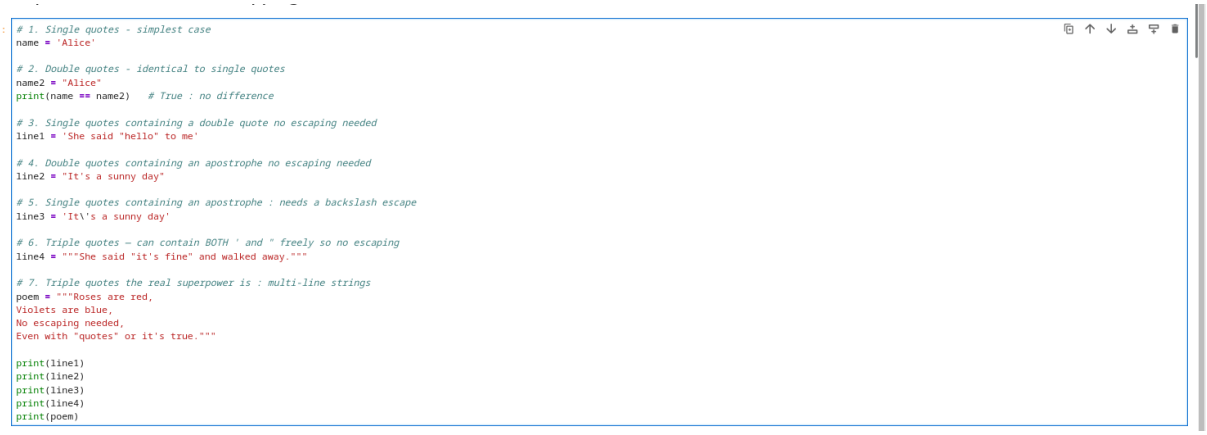
## 2.4 String

**What:** The `str` type represents an array of characters enclosed in single, double, or triple quotes (e.g. `"Hello"`).

**Why:** Text-based data – names, messages, file contents – needs a dedicated type that supports operations like concatenation, slicing, and searching.

**Where and How:** Used for storing and manipulating names, messages, file paths, and any textual data.

Example: `course = "Scientific Computing"`.



```
# 1. Single quotes - simplest case
name = 'Alice'

# 2. Double quotes - identical to single quotes
name2 = "Alice"
print(name == name2)   # True : no difference

# 3. Single quotes containing a double quote no escaping needed
line1 = 'She said "hello" to me'

# 4. Double quotes containing an apostrophe no escaping needed
line2 = "It's a sunny day"

# 5. Single quotes containing an apostrophe : needs a backslash escape
line3 = 'It\'s a sunny day'

# 6. Triple quotes — can contain BOTH ' and " freely so no escaping
line4 = """She said "it's fine" and walked away."""

# 7. Triple quotes the real superpower is : multi-line strings
poem = """Roses are red,
Violets are blue,
No escaping needed,
Even with "quotes" or it's true."""

print(line1)
print(line2)
print(line3)
print(line4)
print(poem)
```

Figure 3: Output of string quotes

## 3 Arithmetic Operators

**What:** Symbols that perform mathematical operations:

- `+` (addition)
- `-` (subtraction)
- `*` (multiplication)
- `/` (true division)
- `//` (floor division)
- `%` (modulus)
- `**` (exponentiation)

**Why:** Almost every program needs to perform calculations, from simple sums to complex formulas, so these operators are required.

**Where and How:** Used in numeric computations such as calculating totals, averages, remainders, or powers.

Example: multiplication `total = price * quantity`.

## 4 Comparison Operators

**What: What:** Operators that compare two values and return a boolean result:

- `==` (equal to)
- `!=` (not equal to)
- `>` (greater than)
- `<` (less than)
- `>=` (greater than or equal to)
- `<=` (less than or equal to)

**Why:** Programs need to make decisions based between two or more values, e.g if one number bigger than another or are two values equal, which is the conditional logic and it outputs true or false, 1 or 0.

**Where and How:** Used inside `if`/`while` conditions to control program flow. Example: `if age >= 18:  print("Eligible to vote")`.

## 5 Assignment Operators

**What:** Operators that assign a value to a variable:

- `=` — assigns the value on the right to the variable on the left
- `+=` — adds and assigns (e.g. `x += 1` means `x = x + 1`)
- `-=` — subtracts and assigns
- `*=` — multiplies and assigns
- `/=` — divides and assigns
- `//=` — floor divides and assigns
- `%=` — takes modulus and assigns
- `**=` — raises to a power and assigns

**Why:** assignment operators provide a way to update a variable based on its current value without avoiding repetition e.g. writing `x += 1` instead of `x = x + 1`.

**Where and How:** Commonly used to update counters inside loops.

Example: `total += value`.

## 6 Type Conversion

**What:** Type conversion is converting a value from one data type to another, using functions like `int()`, `float()`, `str()`, and `bool()`.

**Why:** Data does not always come from user in the type a program needs for example, `input()` always returns a string, so it must be converted before it can be used in arithmetic such as integer

**Where and How:** Used in areas such as converting user input to a number, or converting a number to a string for display. Example: `age = int(input("Enter age:  "))`.

```
[15]: roll_no = int(input("Enter your roll_no: "))
      print(roll_no)
      print(type(roll_no))

      Enter your roll_no:  12
      12
      <class 'int'>
```

Figure 4: Input rollno

# 7 Input and Output

**What:** Input and output (I/O) functions let a program communicate with the user. `input()` reads a line of text typed by the user (as a string), and `print()` displays text or values on the screen.

**Why:** Programs need to receive data from a user and show results back to them as not everything can be hard coded.

**Where and How:** `input()` is used to collect information such as a name or age; `print()` is used to display results, messages, and formatted output.

Example: Figure 4.

# 8 Strings

**What:** A string is an ordered, immutable (i.e cannot change) sequence of characters.

**Why:** We use strings because they are the universal medium for computers to store, understand, and display human language.

**Where and How:** Strings are one way a code can display readable information to a user or collect data back from them.

Everything from button labels to error messages relies on strings.

User Input: Names, passwords, and search queries are taken and stored as strings.

## 8.1 String Creation

Strings are created by enclosing characters in single quotes (`'text'`), double quotes (`"text"`), or triple quotes (`'''text'''` / `"""text"""` for multi-line strings).

Example explanation: Figure 3.

## 8.2 Indexing

**What:** Indexing accesses a single character of a string by its position, using square brackets, starting with `0` as the first index and `-1` as the last, incrementing and decrementing respectively to access the required value.

**Why:** Sometimes only one specific character from a string is needed, not the whole string, so a way to pick out a single character by its position is required.

**How:** Done using square brackets after the string, e.g. `text[0]` gives the first character and `text[-1]` gives the last character.

Example explanation: Figure 5.

## 8.3 Slicing

**What:** Slicing extracts a substring using the syntax `text[start:stop:step]`, where `stop` is exclusive.

**Why:** Often a portion of a string is needed instead of a single character or the entire string, such as the first few letters or the string in reverse order.

**How:** Done by specifying a start and stop index (and optionally a step) inside square brackets, e.g. `text[0:6]` extracts the first 6 characters and `text[::-1]` reverses the string using a step of `-1`.

Example explanation: Figure 5.

### 8.4 String Operations

Common operations used to clean, combine, and analyze text include:

- `+` — concatenation (joins two strings together)
- `*` — repetition (repeats a string a given number of times)
- `in` — containment check (tests if a substring exists inside a string)
- `len()` — length (returns the number of characters)
- `.upper()` — converts all characters to uppercase
- `.lower()` — converts all characters to lowercase
- `.strip()` — removes leading and trailing whitespace
- `.split()` — breaks a string into a list of substrings

Example explanation: Figure 5.

```
[ ]: # --- String Creation ---
s1 = 'Hello'
s2 = "World"
s3 = '''This is a
multi-line string'''
print(s1, s2)
print(s3)

# --- Indexing ---
text = "Python"
print(text[0])    # first character
print(text[-1])   # last character

# --- Slicing ---
print(text[0:4])   # characters 0 to 3
print(text[::-1])  # reversed string

# --- String Operations ---
print(s1 + " " + s2)        # concatenation
print(s1 * 3)                # repetition
print("Py" in text)          # containment check
print(len(text))             # length
print(text.upper())          # uppercase
print(text.lower())          # lowercase
print("  padded  ".strip())  # strip whitespace
print("a,b,c".split(","))    # split
```

Figure 5: String experiments

# 9 Lists

**What:** A list is an ordered, mutable (i.e can chanhe) collection of items, which can be of mixed types, created with square brackets, e.g. `[1, 2, 3]`.

**Why:** Lists let a program group related data together and process it as a single unit, rather than managing many separate variables.

**Where and How:** Used to store collections such as a list of student names, marks, or any sequence of items that may need to grow, shrink, or be reordered.

Example explanation: Figure 6.

## 9.1 Creating Lists

A list is created by placing comma separated values inside square brackets, e.g. `fruits = ["Apple", "Banana", "Mango"]`.

## 9.2 Indexing

Individual elements are accessed by position, e.g. `fruits[0]` for the first item and `fruits[-1]` for the last.

## 9.3 Slicing

A sublist is extracted using `list[start:stop]`, e.g. `fruits[1:3]` returns items at index 1 and 2.

## 9.4 Updating Elements

Because lists are mutable, an element can be changed in place by assigning to its index, e.g. `numbers[1] = 200`, unlike strings which cannot be modified this way.

## 9.5 Nested Lists

**What:** A nested list is a list containing other lists as elements, e.g. `matrix = [[1,2],[3,4]]`.

**Why and How:** Useful for representing grid or table like data such as matrices, where elements are accessed with double indexing, e.g. `matrix[0][1]`.

## 9.6 Basic List Methods

- **append(x):** Adds `x` to the end of the list.
- **insert(i, x):** Inserts `x` at position `i`.
- **remove(x):** Removes the first occurrence of value `x`.
- **pop(i):** Removes and returns the item at index `i` (last item if no index given).
- **sort():** Sorts the list in place in ascending order.
- **reverse():** Reverses the order of the list in place.
- **len():** Returns the number of elements in the list (a built-in function, not a method).

```
colors = ["Red", "Green", "Blue"]
print(colors)

student = ["Adi", 21, 8.5, True]    # string, int, float, boolean together mixed data types
print(student)

print(colors[0])    # Indexing : first item
print(colors[-1])   # last item

print(colors[0:2])  # silicing items at index 0 and 1

scores = [45, 60, 78] #updating
scores[0] = 90
print(scores)

seating = [["A1", "A2"], ["B1", "B2"]] #nested example
print(seating)
print(seating[1][0])   # element accessed at row 1, column 0

marks = [55, 70, 65]

marks.append(80)         # add to end
print("After append:", marks)

marks.insert(0, 40)      # insert 40 at index 0
print("After insert:", marks)

marks.remove(70)         # remove first occurrence of 70
print("After remove:", marks)

popped = marks.pop()     # remove and return last item
print("Popped value:", popped)
print("After pop:", marks)

marks.sort()              # sort ascending
print("After sort:", marks)

marks.reverse()           # reverse order
print("After reverse:", marks)

print("Length of list:", len(marks))
```

Figure 6: List experiments

## 10 Mini Experiments

For each experiment below: the code executed in Jupyter Notebook is shown in notebook in github repository.